22.10.2026

# E 308 – Programmieren 3

Termin 3: Grundelemente der OOP I – Klassen & Objekte

D. Wild

Hochschule Kempten – Fakultät Elektrotechnik

# Agenda

- Wiederholung
- Klasse vs. Objekt
- Attribute & der Konstruktor
- Methoden & Methodenaufruf
- Instanz- vs. Klassenattribute
- `self` vs. `cls`
- Sichtbarkeit in Python
- Anwendung: Die DataSource-Klasse
- Projektstruktur: Module & Pakete
- Praktikum & Ausblick

# Wiederholung

## Termin 2 & Sondertermin Python-Grundlagen

**Termin 2: Versionsverwaltung**

- Zen of Python (PEP 20)
- Git: Commit, Branch, Repository; Merging & Konflikte
- Remote: `clone`, `fetch`, `pull`, `push`
- Feature Branch Workflow (unser Standard)

**Sondertermin: Python-Grundlagen**

- Terminal & `cd`, `.venv`, `requirements.txt`
- Datentypen, Listen, Dictionaries, Schleifen, Funktionen, Lambda
- Eigene Module & `if __name__ == "__main__":`
- Streamlit & Folium live

**Heute** — Aus Funktionen und Dictionaries werden *Klassen*: Daten (Attribute) und die Funktionen dazu (Methoden) in einem Baustein.

# Rückblick: Bauplan vs. Bauteil

## Aus Termin 1

**Definition** — Eine *Klasse* ist der Bauplan; ein *Objekt* (eine *Instanz*) ist ein konkretes, aus diesem Bauplan gebautes Exemplar mit eigenem Zustand.

- Heute: wie schreibt man diesen Bauplan tatsächlich in Python?
- Durchgängiges Beispiel: die Klasse `DataSource` – ein Grundbaustein für jede Folium-/Streamlit-Anwendung

# Eine erste Klasse

## Minimal, aber gültig

```
1 class DataSource:
2     pass
```

**Beispiel** — `pass` bedeutet "hier folgt (noch) nichts" – eine Klasse ohne Inhalt ist syntaktisch gültig, aber noch nicht nützlich. Als Nächstes geben wir ihr Zustand und Verhalten.

# Namen nach PEP 8

## Der offizielle Style Guide für Python-Code

| Was | Schreibweise | Beispiel |
|---|---|---|
| Klassen | `CapWords` | `DataSource`, `WeatherDataSource` |
| Funktionen, Methoden, Variablen, Attribute | `snake_case` | `update_interval_min`, `how_many()` |
| Konstanten | `UPPER_CASE` | `MAX_RETRIES = 3` |
| Module & Pakete | kurz, klein | `weathermap`, `model.py` |
| Nur intern | führender _ | `_cache` |

**Unsere Konvention** — Bezeichner (Klassen, Methoden, Variablen, Module) auf **Englisch**, Kommentare und Ausgabetexte dürfen deutsch sein.

# Attribute

## Der Zustand eines Objekts

**Definition** — *Attribute* sind Variablen, die zu einem Objekt gehören und dessen Zustand beschreiben.

- Unsere Klasse `DataSource` braucht z. B. die Attribute `name` und `update_interval_min`
- Jedes `DataSource`-Objekt hat *seine eigenen* Werte für diese Attribute

# Der Konstruktor `__init__`

## Objekt initialisieren

```python
class DataSource:
    def __init__(self, name, update_interval_min):
        self.name = name
        self.update_interval_min = update_interval_min
```

**Definition** — `__init__` wird *automatisch* aufgerufen, sobald ein neues Objekt erzeugt wird – hier werden die Attribute gesetzt.

# `self` erklärt

## Referenz auf die eigene Instanz

- `self` steht für "dieses konkrete Objekt, auf dem die Methode gerade läuft"
- Python übergibt `self` beim Aufruf *automatisch* – in der *Definition* muss es trotzdem als erster Parameter dastehen

```
weather.status()
# ist unter der Haube äquivalent zu:
DataSource.status(weather)
```

**Achtung** — Beim *Aufruf* `objekt.methode(...)` wird `self` *nicht* mit angegeben – nur bei der *Definition*.

# Objekte erzeugen

## Instanziierung

```
weather = DataSource("Wetterdienst-API", 15)
location = DataSource("Standort-API", 60)
```

**Beispiel** — `weather` und `location` sind zwei unabhängige Objekte derselben Klasse – jedes mit *eigenem* Zustand. Ändert man `weather.name`, bleibt `location.name` unberührt.

# Der Destruktor `__del__`

## Das Gegenstück zu `__init__`

```python
class DataSource:
    def __init__(self, name):
        self.name = name
        print(f"{self.name} erzeugt")

    def __del__(self):
        print(f"{self.name} wird zerstört")


weather = DataSource("Wetterdienst-API")
del weather  # Wetterdienst-API wird zerstört
```

**Definition** — `__del__` wird aufgerufen, kurz bevor Python ein Objekt endgültig entfernt – meist zum Aufräumen (z. B. eine offene Verbindung schließen).

# Destruktoren: Python vs. C++

## Wann läuft `__del__` wirklich?

**Wichtiger Unterschied** — In C++ läuft der Destruktor *deterministisch* – exakt dann, wenn ein Stack-Objekt den Gültigkeitsbereich verlässt oder `delete` aufgerufen wird. In Python entscheidet der *Garbage Collector*, *wann* – das kann sofort sein, verzögert, oder (bei Zyklen) fast beliebig spät.

- `__del__` eignet sich deshalb *nicht* für zeitkritisches Aufräumen wie das Schließen von Dateien oder Netzwerkverbindungen
- Pythonic dafür: ein *Context Manager* (`with ...`) mit garantiertem, sofortigem Aufräumen – mehr dazu bei Bedarf im Praktikum
- In der Praxis kommt `__del__` deutlich seltener vor als `__init__`

# Methoden

## Das Verhalten eines Objekts

```python
class DataSource:
    def __init__(self, name, update_interval_min):
        self.name = name
        self.update_interval_min = update_interval_min

    def status(self):
        return f"{self.name}: alle {self.update_interval_min} min"
```

**Beispiel** — `status()` greift über `self` auf die Attribute *desselben* Objekts zu, auf dem sie aufgerufen wird.

# Methodenaufruf

`objekt.methode()`

```
weather = DataSource("Wetterdienst-API", 15)
print(weather.status()) # Wetterdienst-API: alle 15 min
```

**Definition** — Die Punkt-Notation `objekt.methode(...)` bedeutet: "rufe diese Methode *auf diesem Objekt* auf".

# Instanzattribute vs. Klassenattribute

```python
class DataSource:
    count = 0 # Klassenattribut -- geteilt von allen Instanzen

    def __init__(self, name, update_interval_min):
        self.name = name # Instanzattribut
        self.update_interval_min = update_interval_min
        DataSource.count += 1


source1 = DataSource("Wetterdienst-API", 15)
source2 = DataSource("Standort-API", 60)
print(DataSource.count) # 2
```

**Merken** — Instanzattribute (`self.x`) gehören *einem* Objekt – Klassenattribute (`DataSource.x`) gehören *allen* Instanzen gemeinsam.

# Instanzmethode vs. Klassenmethode

`@classmethod`: **Der erste Parameter ist die Klasse, nicht das Objekt**

```python
class DataSource:
    count = 0                    # Klassenattribut

    def __init__(self, name, update_interval_min):
        self.name = name
        self.update_interval_min = update_interval_min
        DataSource.count += 1

    def status(self):            # Instanzmethode: self = dieses Objekt
        return f"{self.name}: alle {self.update_interval_min} min"

    @classmethod
    def how_many(cls):           # Klassenmethode: cls = die Klasse
        return f"{cls.count} Datenquellen angelegt"

    @classmethod
    def from_config(cls, config): # erzeugt ein neues Objekt
        return cls(config["name"], config["update_interval_min"])
```

# Was übergibt Python automatisch?

## Der erste Parameter hängt von der Methodenart ab

```
weather = DataSource.from_config({"name": "Wetterdienst-API",
                                  "update_interval_min": 15})
# Python ruft auf: from_config(DataSource, {...}) -> cls = DataSource

weather.status()
# Python ruft auf: DataSource.status(weather) -> self = weather

DataSource.how_many() # "1 Datenquellen angelegt"
# Python ruft auf: how_many(DataSource)       -> cls = DataSource
```

- Instanzmethode: Python setzt das Objekt *vor dem Punkt* als `self` ein
- Klassenmethode: Python setzt die *Klasse* als `cls` ein – auch wenn man sie über ein Objekt aufruft (`weather.how_many()`)

# Beispiel: `cls` in der Praxis

## Ein Objekt aus einer Texteingabe erzeugen

```python
class Coordinate:
    def __init__(self, latitude, longitude):
        self.latitude = latitude
        self.longitude = longitude

    @classmethod
    def from_string(cls, text):
        """Erzeugt eine Koordinate aus einem Text wie "47.73, 10.31"."""
        latitude, longitude = text.split(",")
        return cls(float(latitude), float(longitude))


kempten = Coordinate.from_string("47.73, 10.31") # z. B. aus einem Eingabefeld
print(kempten.latitude, kempten.longitude)   # 47.73 10.31
```

- **Warum nicht in `__init__`?** Der Konstruktor bleibt einfach (zwei Zahlen rein), jedes weitere Eingabeformat bekommt eine eigene `from_...`-Methode statt einer `if`-Kaskade.
- **Kennen Sie schon:** `date.fromisoformat("2026-10-15")`, `dict.fromkeys(["a", "b"], 0)`, `pd.DataFrame.from_dict(daten)` – alles Klassenmethoden.

# Wieso gibt es diesen Unterschied?

## Nicht jede Aufgabe gehört zu einem einzelnen Objekt

- **Aufgaben, die alle Objekte betreffen:** „Wie viele Datenquellen gibt es?“ ist keine Frage an `weather`, sondern an die Klasse.
- **Alternative Konstruktoren:** `from_config` soll ein Objekt *erst erzeugen* – es gibt also noch kein Objekt, das als `self` übergeben werden könnte. Vorhanden ist nur die Klasse.
- **Warum `cls` statt fest `DataSource`?** Mit Vererbung (Termin 4) liefert `WeatherDataSource.from_config(...)` automatisch eine `WeatherDataSource` – ohne eine Zeile Code zu ändern.

# `self` oder `cls`? Die Entscheidungsregel

| | **Instanzmethode** | **Klassenmethode** |
|---|---|---|
| Kennzeichen | keiner (Standardfall) | `@classmethod` |
| 1. Parameter | `self` = das Objekt | `cls` = die Klasse |
| Zugriff auf | Instanz- *und* Klassenattribute | nur Klassenattribute |
| Typischer Einsatz | Verhalten eines Objekts: `status()`, `lade_daten()` | alternative Konstruktoren, Zähler, gemeinsame Einstellungen |
| Aufruf | `weather.status()` | `DataSource.how_many()` |

**Faustregel** — Braucht die Methode die Daten *eines* Objekts? → `self`. Braucht sie nur die Klasse, etwa um ein neues Objekt zu bauen? → `@classmethod` mit `cls`.

`self` und `cls` sind Namenskonventionen, keine Schlüsselwörter – was übergeben wird, entscheidet der Decorator. Methoden ganz ohne `self`/`cls` (`@staticmethod`): Termin 4.

# Sichtbarkeit

## public, _protected, __private

```
class DataSource:
    def __init__(self, api_key):
        self.name = "Wetterdienst-API" # public
        self._cache = {}               # protected (Konvention)
        self.__api_key = api_key # private (Name Mangling)
```

- **public** (kein Präfix): von überall zugreifbar
- **_protected** (ein _): Konvention "nur intern verwenden" – technisch nicht verhindert
- **__private** (zwei _): Name Mangling (`_DataSource__api_key`) erschwert Zugriff von außen

# Public & Private in C++

## Zum Vergleich: hart erzwungene Sichtbarkeit

```cpp
class DataSource {
public:
    DataSource(std::string key) : _api_key(key) {}
    std::string name = "Wetterdienst-API"; // public

    std::string api_key() const { // public: kontrollierter Zugriff
        return _api_key;
    }

private:
    std::string _api_key;         // private: nur in der Klasse selbst
};
```

- `q.name = "Standort-API";` und `q.api_key();` – OK, `public`
- `q._api_key = "x";` – **Compilerfehler**, `private` ist von außerhalb der Klasse nicht erreichbar

# Konvention statt Erzwingung

**Anders als in C++** — Python verhindert den Zugriff auf `_protected`/`__private` Attribute technisch *nicht* vollständig – anders als `public`/`private` in C++ oder Java, wo der Compiler den Zugriff blockiert. Die Philosophie: "We are all consenting adults here" – Sichtbarkeit ist eine *Vereinbarung* im Team, keine harte Schranke.

- Quasi-Standard in der Praxis: ein einzelner führender Unterstrich (`_attribut`) – signalisiert "intern, bitte nicht von außen verwenden"
- Der doppelte Unterstrich (Name Mangling) ist die Ausnahme, kein Ersatz für `private`
- Wird später mit `@property` (Termin 5) sauber kombiniert

# Die vollständige DataSource-Klasse

## Alles zusammen

```python
class DataSource:
    count = 0

    def __init__(self, name, update_interval_min, api_key):
        self.name = name
        self.update_interval_min = update_interval_min
        self.__api_key = api_key
        DataSource.count += 1

    def status(self):
        return f"{self.name}: alle {self.update_interval_min} min"


weather = DataSource("Wetterdienst-API", 15, "sk-demo-123")
print(weather.status())
```

# Vom Skript zum Projekt

## Wieso eine Datei nicht reicht

- Ab heute schreiben vier bis sechs Personen gleichzeitig an einem Code – eine einzige `main.py` mit 800 Zeilen erzeugt bei jedem Merge Konflikte
- Jede Klasse (bzw. jede Gruppe eng zusammengehöriger Klassen) bekommt ihre eigene Datei
- Dateien, die thematisch zusammengehören, kommen in einen gemeinsamen Ordner

| **Begriff** | **Was ist das?** | **Import** |
|---|---|---|
| **Modul** | eine `.py`-Datei | `import model` |
| **Paket** | ein Ordner mit Modulen und `__init__.py` | `import weathermap.model` |

Eigene Module kennen Sie aus dem Sondertermin (`wetterdaten.py`) – jetzt ordnen wir sie in Pakete.

# Empfohlene Projektstruktur

## Vorlage für Ihr Team-Repository

```
weathermap/             <- Git-Repository
|-- README.md
|-- requirements.txt
|-- .gitignore
|-- app.py               <- Einstiegspunkt
|-- weathermap/         <- das Paket
|   |-- __init__.py
|   |-- model.py        <- Coordinate, ...
|   |-- view.py
|   `-- datasources/    <- Unterpaket
|       |-- __init__.py
|       |-- base.py     <- DataSource
|       |-- weather.py
|       `-- location.py
`-- tests/
    `-- test_datasources.py
```

- `app.py` im Wurzelordner startet die Anwendung: `python app.py` bzw. `streamlit run app.py`
- Der eigentliche Code liegt im Paket – gleicher Name wie das Projekt, nur Kleinbuchstaben und Unterstriche
- Tests getrennt vom Code (Termin 10)
- Immer aus dem *Wurzelordner* heraus starten

# Was macht `__init__.py`?

**Aufgabe** — Eine Datei `__init__.py` macht einen Ordner zu einem *regulären Paket*. Sie wird beim *ersten* Import des Pakets einmal ausgeführt – und ist meistens leer.

**Wann brauche ich sie?**

- In *jedem* Ordner, aus dem Sie importieren – auch in Unterordnern
- *Nicht* im Wurzelordner des Repositorys und nicht in `tests/`

**Was darf hinein?**

- Nichts (Normalfall)
- Imports, die das Paket nach außen anbietet (nächste Folie)
- *Keine* Logik, keine Ausgaben, keine Netzwerkzugriffe

# `__init__.py` als Schaufenster

## Kurze Imports für die Nutzer des Pakets

```python
# weathermap/datasources/__init__.py
from .base import DataSource
from .weather import WeatherDataSource
from .location import LocationDataSource

__all__ = ["DataSource", "WeatherDataSource", "LocationDataSource"]
```

**Ohne**

```python
from weathermap.datasources.weather \
    import WeatherDataSource
```

**Mit**

```python
from weathermap.datasources \
    import WeatherDataSource
```

Die interne Aufteilung in Dateien kann sich später ändern, ohne dass Code außerhalb des Pakets angepasst werden muss – Kapselung auf Paketebene. `__all__` legt fest, was `from ... import *` liefert.

# Absolute & relative Imports

```
# weathermap/datasources/weather.py
from .base import DataSource           # relativ: gleicher Ordner
from ..model import Coordinate         # relativ: eine Ebene hoeher
from weathermap.model import Coordinate # absolut: ab Paketwurzel
```

- **Absolut** ist eindeutig und überall lesbar – in `app.py` und in Tests immer absolut importieren
- **Relativ** (`.` = dieser Ordner, `..` = eine Ebene höher) ist innerhalb eines Pakets bequem – im Team *einen* Stil festlegen

**Typischer Fehler** — Direkt gestartet: `python weathermap/datasources/weather.py`
⇒ `ImportError: attempted relative import with no known parent package`
Richtig, als Modul: `python -m weathermap.datasources.weather`

# Sprint 1: Stand der Dinge

**Iteration 1 läuft seit dem Sondertermin am 15.10.**

- **Stand prüfen:** Ist die Spike-Story erledigt? Was steht in `Ready`, was in `In Progress`?
- **Nachschärfen:** Neu dazu kommen die ersten Klassen & Objekte von heute – passende Tasks zu den Stories ergänzen
- **Tasks verteilen:** Jede Story hat Tasks und eine verantwortliche Person, Feld `Iteration 1`
- **Gruppe B:** Sprint 1 endet am 28.10. – erster Sprintwechsel gemeinsam in Ihrer Übung am 29.10.
- **Gruppe A:** Sprint 1 läuft bis 04.11. – erster Sprintwechsel gemeinsam am 05.11.

**Zeitrahmen** — Kurz halten – ca. 10 Minuten, der Rest der Zeit gehört der Umsetzung.

# Übung: Gruppe A heute, Gruppe B am 29.10.

## Sprint 1 – Ans Projekt angepasste Basisklassen implementieren

- Codestruktur in der Gruppe überlegen, sodass Datenzugriffe etc. als Klassen implementiert werden können
- `Application`-Basisklasse: Attribute `name`, `version`, Liste `data_sources`; Methode `status()`
- Beide Klassen im Python-Interpreter ausprobieren, mehrere Objekte erzeugen – passend zu Ihrem eigenen Projektthema
- **Anschließend:** Projektstruktur (Paket, `__init__.py`, `app.py`) per Pull Request anlegen – jede Klasse in ihrem eigenen Modul

# Zusammenfassung

**Kernaussagen —**

- `__init__` initialisiert ein Objekt beim Erzeugen
- `__del__` läuft beim Zerstören, aber *nicht deterministisch* – kein Ersatz für Context Manager bei zeitkritischem Aufräumen
- Instanzebene vs. Klassenebene: `self.x` und `self` gehören einem Objekt, Klassenattribute und `cls` (`@classmethod`) der ganzen Klasse
- Namen nach PEP 8 und auf Englisch (`DataSource`, `update_interval_min`); `_`/`__` sind Konvention, keine Erzwingung
- Modul = Datei, Paket = Ordner mit `__init__.py`; immer aus dem Wurzelordner starten
- Nächste Woche: Vererbung & Polymorphismus

# Fragen?

E 308 – Programmieren 3 – Wintersemester 2026/27